AthleteIQ

Multi-Sport Athlete Database Management System

Database Lab Project

Group : AthleteIQ

Member 1: Ismail Khan

Member 2: Rayan Alam

Repository:

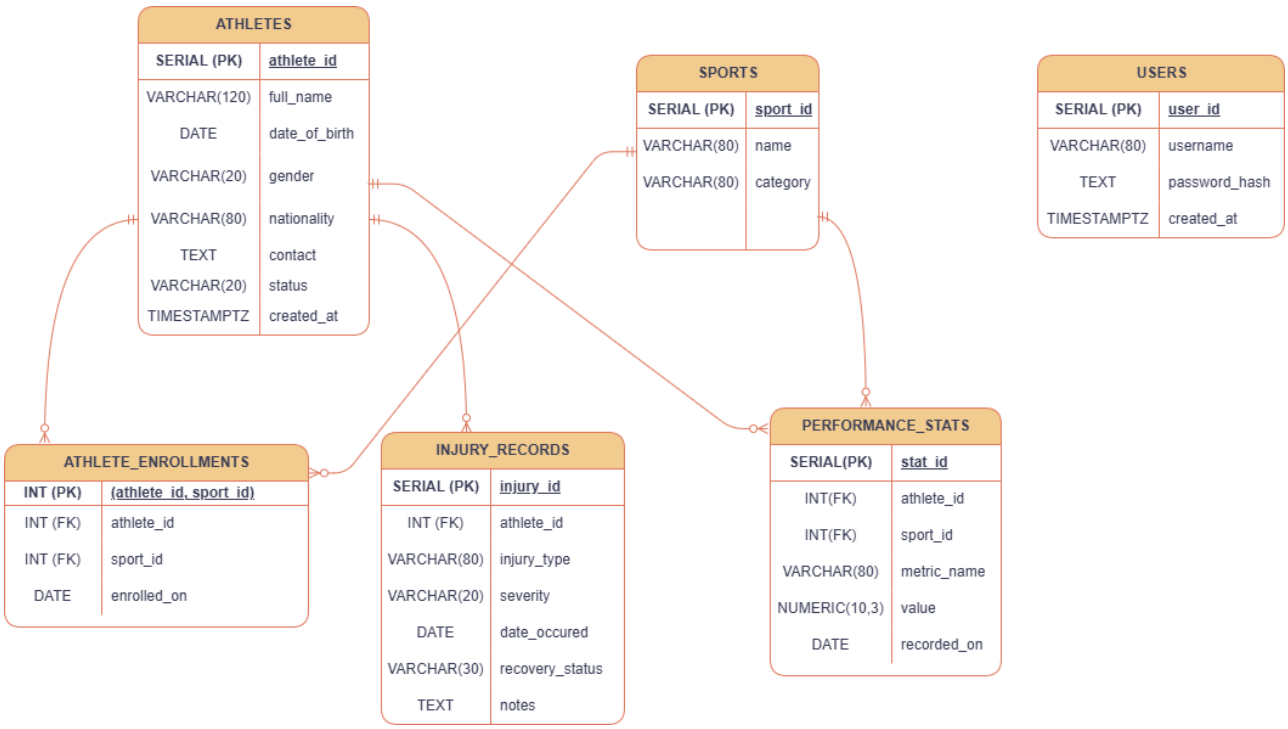
https://github.com/ismailkhan1707/athleteIQ.git

LAB Instructor: Mr. Ali Hasan

School of Computer Sciences & IT

Institute of Management Sciences, Peshawar

1. Entity Relationship Diagram (ERD)



2. Normalization Walkthrough

Tables in Schema

- ATHLETES
- SPORTS
- USERS
- ATHLETE_ENROLLMENTS
- INJURY_RECORDS
- PERFORMANCE_STATS

First Normal Form (1NF)

All six tables satisfy 1NF. Every column across all tables holds atomic, single values with no repeating groups or multi-valued attributes. Each table has a defined primary key — athlete_id, sport_id, user_id, injury_id, stat_id, and a composite (athlete_id, sport_id) in ATHLETE_ENROLLMENTS.
Result: No changes were needed.

Second Normal Form (2NF)

Five of the six tables have single-column primary keys, making partial dependencies impossible by definition. The only composite key exists in ATHLETE_ENROLLMENTS, where the sole non-key attribute enrolled_on depends on both athlete_id and sport_id together — representing the date a specific athlete enrolled in a specific sport. No partial dependency exists.
Result: No changes were needed.

Third Normal Form (3NF)

ATHLETES:

All attributes (full_name, date_of_birth, gender, nationality, contact, status, created_at) describe the athlete directly and depend solely on athlete_id. No transitive dependencies exist.

SPORTS:

Both name and category depend directly on sport_id. category is a simple descriptive field with no attributes of its own, so no transitive dependency arises.

USERS:

username, password_hash, and created_at all depend directly on user_id. No attribute is derived from another non-key column.

ATHLETE_ENROLLMENTS:

The only non-key attribute is enrolled_on, which depends directly on the composite key (athlete_id, sport_id). No transitive dependency is possible.

INJURY_RECORDS:

All attributes (injury_type, severity, date_occured, recovery_status, notes) describe the specific injury instance and depend directly on injury_id. No transitive dependency is possible.

PERFORMANCE_STATS:

metric_name, value, and recorded_on all depend directly on stat_id. athlete_id and sport_id appear as foreign keys, not as sources of transitive dependency.

Result:

No changes were needed across any table.

Summary

1NF — Satisfied — No changes made
2NF — Satisfied — No changes made
3NF — Satisfied — No changes made
The schema was already fully normalized to 3NF as designed.

3. DataFlow Description

Data Entry Points

Data enters the AthleteIQ system through the following sources:

Admin Login: USERS represents application-level admin accounts. These are individuals who log into the AthleteIQ system using their credentials to manage athletes, sports, enrollments, and records. They are not coaches or managers but system administrators with access to the application interface.

Athlete Registration: When a new athlete joins the system, their personal information (name, date of birth, gender, nationality, contact, status) is recorded in the ATHLETES table.

Sport Enrollment: Coaching staff enroll athletes into specific sports, creating records in ATHLETE_ENROLLMENTS that link an athlete to a sport with an enrollment date.

Injury Reporting: Medical staff or coaches log injury incidents for athletes, writing to the INJURY_RECORDS table with details on injury type, severity, date, and recovery status.

Performance Logging: After training sessions or matches, coaches or analysts record performance metrics (goals scored, pass accuracy, sprint speed, etc.) for each athlete per sport, writing to the PERFORMANCE_STATS table.

Data Flow Through the Database

The tables have a clear dependency order that data must respect:

USERS (independent — application admin accounts for system login)

SPORTS (independent — reference table of available sports)

ATHLETES (independent — registered athletes)

ATHLETE_ENROLLMENTS depends on ATHLETES (athlete_id) depends on SPORTS (sport_id)

INJURY_RECORDS depends on ATHLETES (athlete_id)

PERFORMANCE_STATS depends on ATHLETES (athlete_id) depends on SPORTS (sport_id)
Load order for data population: SPORTS and ATHLETES must be populated before

ATHLETE_ENROLLMENTS, INJURY_RECORDS, or PERFORMANCE_STATS can reference them. USERS is independent and can be loaded at any point.

Data Outputs

Once data is loaded, the system supports the following query outputs:

Athlete Profile Report —Joins ATHLETES with ATHLETE_ENROLLMENTS and SPORTS to show which sports an athlete is enrolled in.

Performance Summary — Queries PERFORMANCE_STATS filtered by athlete_id and sport_id to return recorded metrics over time (e.g., goal scoring trend across football sessions).

Injury History Report — Queries INJURY_RECORDS by athlete_id to display a timeline of injuries, severity levels, and recovery status for medical staff review.

Sport Roster — Joins ATHLETE_ENROLLMENTS with ATHLETES to list all athletes enrolled in a given sport (e.g., all Football players).

Active vs Inactive Athletes — Filters ATHLETES by status to monitor which athletes are currently active in the system.